

Engineering Learnings

What building a DRC-repair agent taught us

Companion to the **ASU problem** deck — the research and experiments behind the design.

Built in one focused day; every claim is measured and reproducible.

Harikrishnan KC · Team **Chip Convergence** · greatharikrishnan@gmail.com

1 · Version-exactness is a first-class requirement

The evaluator is a **string-equality gate**: `klayout -v` must read exactly `0.30.1`, or DRC is refused. macOS Homebrew gives 0.30.9 (and Gatekeeper flags the unsigned app).

We initially fought the macOS install; the right move was to **stop changing the host and go Docker** — an amd64 image with the pinned `.deb`, run under emulation on Apple Silicon. The payoff wasn't just "it runs": our container DRC matches the reference report on **11/14 rules exactly**, proving our environment *is* the scoring environment.

Lesson: for a benchmark scored in a pinned environment, reproduce that environment exactly before trusting a single local number. Docker over host, every time.

2 · Measure with the scorer's own code, or don't trust the number

Our first instinct was to parse DRC reports and count violations ourselves. Instead we **import the official** `evaluate_repair.py` **functions** — the same render, DRC, count, and connectivity routines the organizers score with.

Why it mattered immediately: feeding the *unmodified* script back scored `final_violation_rate` = 1.29, not 1.0. A per-rule diff (11/14 exact; 3 grid rules inflated by clean 2×/4× factors) localized the gap to a script-rebuild multiplicity artifact — a diagnosis we could only trust because our counter *was* their counter.

Lesson: an inner-loop metric that isn't byte-identical to the scoring metric will optimize the wrong thing. Reuse the scorer.

3 · Read the rule deck, not the error message

We spent a first pass reverse-engineering fixes from the DRC report's human descriptions ("V2 must be the same width as M3..."). Every fix regressed.

The descriptions are lossy. The **actual** `asap7.lydrc` **rule** is precise:

`V2.M3.AUX.2` is satisfied iff the via is *inside* M3 **and has ≥ 2 edges coincident with M3's edges** — and it is coupled to `V2.M2.EN.1` (5 nm enclosure) and `V2.AUX.1` (containment). Once we read the deck, the fix became *derivable* instead of guessable.

Lesson: the authoritative spec is the checker's source, not its output. Read it.

4 · Research reframes the whole problem

One web search was not enough for a specialized domain. Digging into **LAD and ISPD** surfaced the reframe:

- **DRC-Coder (ISPD'25, NVIDIA)** — vision + LLM interpret DRC rules at F1=1.0. Validates the untapped **screenshot** lever (Gemini is multimodal).
- **Wide-metal via enclosure** (PDK/patents) — a *wide* metal needs a *wide* via, not a min-via. That is exactly our 181 via-width-match violations: V2 (min) sitting in a wide M3.
- **MDPI 2025** — standard-cell DRC repair as **simulated annealing over a conflict graph**: the formal model for what we'd found empirically.

Lesson: the literature had already named our problem. An hour of reading changed the plan more than a day of coding.

5 · We reshaped the wrong thing first (and measured our way out)

Every fix that touched the **metal** broke a neighbour — measured, not asserted:

fix	fixes	breaks	net
grow via → metal width	AUX.2	lower-metal enclosure	315→387
shrink metal → via width	AUX.2	upper-via enclosure	315→339
surgical M3 neck	AUX.2 (x-clear)	M3.S.4 (neck shoulders)	315→339

We almost concluded "block repair is global legalization." **The escape was to stop reshaping the metal and reshape the via:** the seeding had split each landing into a multi-cut min-via array; replacing it with one continuous **via bar** (min thickness → no metal widening) fixed the class cleanly — **315 → 178, FVR 0.73**, all 5 blocks 0.68–0.76.

Lesson: a run of failing transforms is not proof of impossibility. It usually means you're moving the wrong object. Exhaust the *object*, not just the *parameters*.

6 · The review loop caught our premature give-up

After the metal-neck was falsified, our own recommendation was "**consolidate and ship the honest negative result.**" An independent review pushed back — the via-bar hypothesis was untested — and it was **right**. One experiment turned a negative result into a win on all five blocks.

Two design choices made it safe to keep swinging:

- **Verify == the scorer** (we import the official evaluator's functions) — no metric drift.
- **Keep-best + a rendered-connectivity credibility gate** — a candidate that regresses OR only *looks* connected (the static checker is source-based) is discarded. A model that over-corrected 315→13,823 never shipped.

Lesson: make the bad outcome unshippable, then let others attack your conclusions. The verification spine is what lets a review say "try again" without risk — and what turns a good review into a real result.

Harikrishnan KC · Chip Convergence · greatharikrishnan@gmail.com